Тема 5. Програмиране на задачи с цикли. Оператори for, while, do...while. Оператори break и continue.

Цел: Съставяне, изпълнение и анализ на C++ програми с повторение, обработка на последователности, валидиране на вход, вложени цикли и итерационни изчисления.

1. Основни конструкции за цикъл

Когато е необходимо да се наруши естествения ред на изпълнение на операторите (във функция) и да се организира многократно изпълнение на оператор/блок от оператори се прилагат три типови циклични програмни конструкции, известни като оператори за цикъл.

Цикълът повтаря оператор или блок от оператори, докато е изпълнено определено условие. Трите основни конструкции в C++ са for, while и do...while. В тялото на цикъла може да има други проверки или цикли.

1.1. Цикъл for — повторения с брояч

```
for (инициализация; условие; актуализация) {
   // тяло на цикъла
}
```

Инициализацията се изпълнява еднократно. Условието се проверява преди всяка итерация и цикълът продължава, докато то е true. Актуализацията се изпълнява след тялото. Ако условието е false още в началото, тялото няма да се изпълни. Пропуснатото условие се приема за true, а пропуснатите други две секции не извършват действие.

Пример: сума на целите числа от 1 до 100 в прав и обратен ред. Променливата sum се дефинира преди цикъла, за да бъде достъпна и след него.

```
int sum = 0;
for (int i = 1; i <= 100; i++) sum += i;

sum = 0;
for (int i = 100; i >= 1; i--) sum += i;
```

1.2. Цикъл while — повторение с предусловие

```
while (условие) {
   // тяло на цикъла
}
```

Условието се проверява преди всяка итерация, затова тялото може да не се изпълни нито веднъж. Конструкцията е удобна, когато броят на повторенията не е известен предварително — например въвеждане до специална стойност или достигане на зададена точност.

```
int i = 1, sum = 0;
while (i <= 100) {
   sum += i;
   i++;
}
```

1.3. Цикъл do...while — повторение с постусловие

```
do {
   // тяло на цикъла
} while (условие);
```

Тялото се изпълнява поне веднъж, защото условието се проверява в края. Цикълът е удобен за менюта и повторно въвеждане на данни до удовлетворяване на условие.

```
int i = 1, sum = 0;
do {
   sum += i;
   i++;
} while (i <= 100);
```

2. Управление на повторенията: break, continue и return

Операторът break прекъсва най-близкия обхващащ цикъл (или switch) и продължава с първия оператор след него. Прилага се при възникване на ситуации, налагащи промяна в обработката на останалите елементи, например от масив или входни данни. Когато цикълът се намира във функция, може да се използва оператор return за прекъсване на цикъла и връщане на управлението в извикващата функция.

```
int sum = 0;
for (int i = 1; ; ++i) {
   sum += i;
   if (i == 100) break;
}
```

Операторът continue се използва само с операторите за цикъл for, while, do...while. Препраща управлението в края на текущата итерация на цикъла и цикличният процес продължава със следващата итерация. Осигурява пропускането на част от операторите в тялото на цикъла, обикновено като резултат от проверка на условие.

При for се изпълнява актуализацията, след което се проверява условието; при while и do...while управлението преминава към проверката на условието. Внимавайте да не пропуснете необходима актуализация на брояч при while.

```
int sum = 0;
for (int i = 1; i <= 10; ++i) {
   if (i == 7 || i == 8) continue;
   sum += i;  // Сумира всички освен 7 и 8.
}
```

return прекратява текущата функция; return 0; в main() означава успешно приключване, а ненулев код обикновено сигнализира грешка. За прекратяване на цялата програма съществува и std::exit() от <cstdlib>, която прекъсва изпълнението на програмата като връща код – 0 (към операционната система) за нормално завършване или ненулева стойност за индикация на грешка.

3. Вложени цикли и надеждно въвеждане

Вложен цикъл е цикъл, разположен в тялото на друг. Вътрешният цикъл се изпълнява за всяка съответна итерация на външния. Обикновено двата цикъла използват различни броячи.

```
for (int row = 1; row <= 3; ++row) {
   for (int col = 1; col <= 4; ++col) {
      std::cout << row * col << " ";
   }
   std::cout << "\n";
}
```

4. Задачи за самостоятелна работа

Задача 5.1. Да се състави C/C++ програма, за въвеждане на последователност от цели числа до въвеждането на числото 0 и намиране на произведението само от положителните числа. Да се реализира в няколко варианта.

Задача 5.2. Да се състави програма, която изчислява стойностите на функцията $f(x)=x^4+2x^3-13x^2+8x$ за стойности на аргумента x, изменящи се в диапазона $0,5 \le x \le 2,5$ със стъпка 0,1

Задача 5.3. Да се състави C/C++ програма, чрез която по зададен интервал от кодове на ASCII символи да се изведат техните изображения в колона на екрана.

Задача 5.4. Съставете програма за намиране големината на ъглите на триъгълник и грешката при тяхното пресмятане, ако са зададени от клавиатура размери на страните – a, b и c. Включете проверка за валидност на въведените стойности.

Задача 5.5. Да се състави C/C++ програма с меню за избор на математическа функция: логаритъм, експонента, квадратен корен или тригонометрична функция: sin(), cos(), tg(), cotg(), както и за пресмятане и извеждане на стойността ѝ по въведен от клавиатурата аргумент a.

Задача 5.6. Да се състави C/C++ програма за пресмятане и извеждане на произведението R за зададен брой множители – n:

$R=(x^2/2+1)(x^3/4+1)(x^4/6+1) \ldots (x^{n+1}/2n+1)$

Задача 5.7. Сумиране на степенен ред до точност 10^{-7}

Да се пресметне $V = a + (x/a) + (x/a)^3 + (x/a)^5 + \ldots$ до достигане на точност 10^{-7}. Нека $q=x/a$. Редът има крайна сума само при $|q|<1$; при $a=0$ изразът е недефиниран. Тъй като всяко следващо събираемо е предишното, умножено по q^2, избягваме повторно извикване на pow(). Използваме do...while и ограничение за броя

Задача 5.8. Моделирайте програмно играта на „топло – студено“ чрез генериране на случайно число в диапазона между 1 и M. Задачата е да познаете генерираното число, като всеки път когато въведете число, програмата ви насочва дали вашето число е по-голямо от него – „топло“ или по-малко от генерираното число – „студено“.

Задача 5.9. Съставете програма за табулиране на функцията V(x), която представлява корен 3-ти от x, чрез итерационната формула $V_i = V_{i-1} + 1/3\cdot(x/V_{i-1}^2 - V_{i-1})$, за стойности на x от −10 до +10, с точност 10^{-5}. Начално приближение за V(x) = x/3, ако x >= 1 и V(x) = x, ако x < 1.

Задача 5.10. Съставете програма за кодиране на текст, като заместите в него символа ‘A’ с ‘B’, символа ‘B’ с ‘C’ и т.н., ‘Z’ с ‘A’.

Задача 5.11. Да се състави C/C++ програма за пресмятане факториелите на числата от 5 до 15 по формулата: N!=1*2*3*...*(N-1)*N, като се прескочат 10! и 13!.

5. Допълнителни въпроси и задачи за самостоятелна работа

1. Какви са разликите между for, while и do...while? Кой цикъл задължително изпълнява тялото поне веднъж?
2. Колко пъти ще се изпълни тялото на while и do...while, ако условието е false още в началото?
3. За какво служат break и continue? Как се държат при вложени цикли?
4. Какво представлява вложен цикъл? Дайте пример с два брояча.
5. Възможно ли е инициализиране и актуализиране на няколко променливи в for?
6. Напишете програма за въвеждане на числа от 0 до 99 до символ *. Изведете сумата и произведението на въведените числа, броя на четните и нечетните числа и средноаритметичната стойност. Предвидете празна последователност и препълване.
7. Въведете оценки в точки (0–100), като отрицателна стойност приключва въвеждането. Проверявайте валидността на всяка оценка. Изведете броя на оценките, минимум, максимум и броя на оценките в интервал, зададен от потребителя.
8. Табулирайте y(x) за x от −100 до 100 със стъпка 10: при $x<0$, $y=\sqrt{(x^2+1)}$; при $x\geq 0$, $y=(x+10)/(x-20)$. Обработете случая x=20, когато функцията не е дефинирана.
9. Напишете програма, която отделя и сумира цифрите на неотрицателно цяло число (unsigned long long).
10. Напишете програма, която брои всички въведени символи до EOF, включително интервали и нови редове.